HOTEL RESERVATION DATABASE MANAGEMENT SYSTEM

Database Design, SQL CRUD, Indexing, Views and Window Functions

Project submitted by	Nikhil Singh
Roll number	2501420049
Database platform	MySQL 8.0+
Project area	Database Management Systems

PROJECT REPORT

A practical relational database solution for managing hotel rooms, guests, reservations, room rates, billing and front-desk operations.

1. Introduction

The Hotel Reservation Database Management System stores hotel room details, guest profiles, reservations, room-rate history and additional charges. It supports day-to-day front-desk work while providing SQL queries for availability, billing, revenue analysis and guest loyalty reporting.

Objectives

• Design a normalized relational schema with primary keys, foreign keys and validation constraints.

• Implement Create, Read, Update and Delete (CRUD) operations for guests and reservations.

• Prevent overlapping reservations for the same room and check availability for a requested date range.

• Calculate nightly room charges, extra charges and total bills.

• Use indexes, EXPLAIN, views, WITH CHECK OPTION and MySQL 8 window functions.

Assumptions and business rules

• A stay uses a check-in date inclusive and a check-out date exclusive. A room is occupied when an existing reservation satisfies existing.check_in < requested.check_out AND existing.check_out > requested.check_in.

• Only reservations with status CONFIRMED or CHECKED_IN block availability. CANCELLED and CHECKED_OUT reservations do not block future dates.

• One reservation is for one room. A reservation can have multiple extra-charge records.

• Rates are stored by room type and effective date. The example billing query uses the rate effective on the check-in date; production systems may calculate each night separately if rates change during a stay.

• All date and monetary examples use MySQL syntax and DECIMAL for currency precision.

2. Relational Schema

The schema uses six core tables. Room types describe categories; rooms are physical inventory; guests hold contact details; reservations connect guests to rooms; room_rates store dated pricing; extra_charges record services added to a reservation.

```
CREATE DATABASE hotel_reservation;
USE hotel_reservation;

CREATE TABLE room_types (
    room_type_id INT AUTO_INCREMENT PRIMARY KEY,
    type_name VARCHAR(50) NOT NULL UNIQUE,
    description VARCHAR(255),
    max_occupancy INT NOT NULL CHECK (max_occupancy > 0)
);

CREATE TABLE rooms (
    room_id INT AUTO_INCREMENT PRIMARY KEY,
    room_number VARCHAR(10) NOT NULL UNIQUE,
    room_type_id INT NOT NULL,
    floor_no INT,
    room_status ENUM('ACTIVE','MAINTENANCE') NOT NULL DEFAULT 'ACTIVE',
    FOREIGN KEY (room_type_id) REFERENCES room_types(room_type_id)
);

CREATE TABLE guests (
    guest_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(120) NOT NULL,
    email VARCHAR(150) NOT NULL,
    phone VARCHAR(25),
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT uq_guest_email UNIQUE (email)
);

CREATE TABLE reservations (
    reservation_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    guest_id INT NOT NULL,
    room_id INT NOT NULL,
    check_in_date DATE NOT NULL,
    check_out_date DATE NOT NULL,
```

```
    booking_status ENUM(
        'CONFIRMED','CHECKED_IN','CHECKED_OUT','CANCELLED'
    ) NOT NULL DEFAULT 'CONFIRMED',
    booked_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CHECK (check_out_date > check_in_date),
    FOREIGN KEY (guest_id) REFERENCES guests(guest_id),
    FOREIGN KEY (room_id) REFERENCES rooms(room_id)
);

CREATE TABLE room_rates (
    rate_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    room_type_id INT NOT NULL,
    effective_from DATE NOT NULL,
    effective_to DATE,
    nightly_rate DECIMAL(10,2) NOT NULL CHECK (nightly_rate >= 0),
    FOREIGN KEY (room_type_id) REFERENCES room_types(room_type_id),
    CHECK (effective_to IS NULL OR effective_to > effective_from),
    INDEX idx_rate_type_dates (room_type_id, effective_from, effective_to)
);

CREATE TABLE extra_charges (
    charge_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    reservation_id BIGINT NOT NULL,
    charge_description VARCHAR(120) NOT NULL,
    amount DECIMAL(10,2) NOT NULL CHECK (amount >= 0),
    charged_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (reservation_id)
        REFERENCES reservations(reservation_id) ON DELETE CASCADE
);
```

Relationship summary: one room type has many rooms and many rate periods; one guest can make many reservations; one room can appear in many reservations over time; one reservation can contain many extra charges.

3. Sample Data

```
INSERT INTO room_types (type_name, description, max_occupancy) VALUES
('Standard', 'Comfortable standard room', 2),
('Deluxe', 'Larger room with upgraded amenities', 3),
('Suite', 'Premium suite', 4);

INSERT INTO rooms (room_number, room_type_id, floor_no) VALUES
('101', 1, 1), ('102', 1, 1),
('201', 2, 2), ('301', 3, 3);

INSERT INTO guests (full_name, email, phone) VALUES
('Aarav Sharma', 'aarav@example.com', '9876500001'),
('Meera Patel', 'meera@example.com', '9876500002'),
('Kabir Verma', 'kabir@example.com', '9876500003');

INSERT INTO room_rates
(room_type_id, effective_from, effective_to, nightly_rate) VALUES
(1, '2026-01-01', NULL, 2500.00),
(2, '2026-01-01', NULL, 4000.00),
(3, '2026-01-01', NULL, 6500.00);

INSERT INTO reservations
(guest_id, room_id, check_in_date, check_out_date, booking_status)
VALUES
(1, 1, '2026-11-12', '2026-11-15', 'CONFIRMED'),
(2, 3, '2026-11-13', '2026-11-16', 'CONFIRMED');

INSERT INTO extra_charges (reservation_id, charge_description, amount)
VALUES (1, 'Breakfast', 900.00), (1, 'Laundry', 350.00);
```

4. CRUD Operations

Create guest and reservation

```
-- Create
INSERT INTO guests (full_name, email, phone)
VALUES ('Riya Das', 'riya@example.com', '9876500004');

-- Create a reservation only after checking availability.
INSERT INTO reservations
(guest_id, room_id, check_in_date, check_out_date, booking_status)
VALUES (4, 2, '2026-11-20', '2026-11-23', 'CONFIRMED');
```

Read guest and reservation information

```
SELECT guest_id, full_name, email, phone
FROM guests
```

```
ORDER BY full_name;

SELECT r.reservation_id, g.full_name, rm.room_number,
       r.check_in_date, r.check_out_date, r.booking_status
FROM reservations r
JOIN guests g ON g.guest_id = r.guest_id
JOIN rooms rm ON rm.room_id = r.room_id
ORDER BY r.check_in_date;
```

Update guest and reservation

```
UPDATE guests
SET phone = '9876500099'
WHERE guest_id = 4;

UPDATE reservations
SET check_in_date = '2026-11-21',
    check_out_date = '2026-11-24'
WHERE reservation_id = 3;
```

Delete / cancel

For business records, cancellation is generally safer than deleting a reservation. The DELETE example is included to demonstrate CRUD; use it only when deletion is permitted by the hotel's retention policy.

```
-- Preferred operational action:
UPDATE reservations
SET booking_status = 'CANCELLED'
WHERE reservation_id = 3;

-- Physical deletion example:
DELETE FROM reservations
WHERE reservation_id = 3;
```

5. Availability, Booking Charges and Room Assignment

Check room availability for a date range

The overlap rule is the central availability test. The comparison uses a half-open interval, so a guest checking out on a date does not block another guest checking in on that same date.

```
SET @wanted_check_in = '2026-11-20';
SET @wanted_check_out = '2026-11-23';

SELECT rm.room_id, rm.room_number, rt.type_name
FROM rooms rm
JOIN room_types rt ON rt.room_type_id = rm.room_type_id
WHERE rm.room_status = 'ACTIVE'
  AND NOT EXISTS (
      SELECT 1
      FROM reservations r
      WHERE r.room_id = rm.room_id
        AND r.booking_status IN ('CONFIRMED','CHECKED_IN')
        AND r.check_in_date < @wanted_check_out
        AND r.check_out_date > @wanted_check_in
  )
ORDER BY rm.room_number;
```

Calculate total booking charges

```
SELECT r.reservation_id, g.full_name, rm.room_number,
       DATEDIFF(r.check_out_date, r.check_in_date) AS nights,
       rt.type_name,
       rr.nightly_rate,
       DATEDIFF(r.check_out_date, r.check_in_date)
           * rr.nightly_rate AS room_charges,
       COALESCE(ec.extra_total, 0) AS extra_charges,
       DATEDIFF(r.check_out_date, r.check_in_date)
           * rr.nightly_rate + COALESCE(ec.extra_total, 0) AS total_bill
FROM reservations r
JOIN guests g ON g.guest_id = r.guest_id
JOIN rooms rm ON rm.room_id = r.room_id
JOIN room_types rt ON rt.room_type_id = rm.room_type_id
JOIN room_rates rr
  ON rr.room_type_id = rt.room_type_id
 AND rr.effective_from <= r.check_in_date
 AND (rr.effective_to IS NULL OR rr.effective_to > r.check_in_date)
LEFT JOIN (
    SELECT reservation_id, SUM(amount) AS extra_total
    FROM extra_charges
```

```
    GROUP BY reservation_id
) ec ON ec.reservation_id = r.reservation_id;
```

This example applies the rate effective at check-in to every night. If a rate can change during a stay, calculate the price night-by-night by joining each stay date to its effective rate period.

Assign or reassign a room

```
-- Assign a room by creating the reservation after the availability check.
INSERT INTO reservations
(guest_id, room_id, check_in_date, check_out_date, booking_status)
VALUES (2, 2, '2026-12-01', '2026-12-04', 'CONFIRMED');

-- Reassign an existing reservation: first verify the target room
-- has no overlapping active reservation, then update it.
UPDATE reservations
SET room_id = 4
WHERE reservation_id = 5;
```

Important: MySQL does not provide an exclusion constraint for date-range overlaps. In a multi-user production system, run the availability check and insert/update in a transaction with an appropriate locking strategy (for example, lock the target room row with SELECT ... FOR UPDATE) so two simultaneous requests cannot book the same room.

6. Indexing and EXPLAIN

The requested composite index supports searches that filter reservations by room and date boundaries. The guest email index supports fast lookup and enforces uniqueness.

```
CREATE INDEX idx_res_room_dates
ON reservations (room_id, check_in_date, check_out_date);

CREATE UNIQUE INDEX uq_guests_email
ON guests (email);

-- Example lookup with an index:
EXPLAIN
SELECT reservation_id
FROM reservations
WHERE room_id = 1
  AND check_in_date < '2026-11-23'
  AND check_out_date > '2026-11-20';

-- Compare with a query that has no useful leading-key filter:
EXPLAIN
SELECT reservation_id
FROM reservations
WHERE check_out_date > '2026-11-20';
```

How to interpret EXPLAIN

• key shows the index MySQL selected. A NULL key can indicate that no index was selected.

• type describes access: const/eq_ref/ref/range are commonly more selective than a full table scan (ALL).

• rows is the optimizer's estimated number of rows examined; it is an estimate, not necessarily the actual count.

• Extra may show Using where, Using index or Using filesort. The exact plan depends on table size, data distribution, statistics and MySQL version.

The composite index is ordered by room_id first, then check_in_date, then check_out_date. It is useful when room_id is constrained and the date range can narrow the scan. Because check_out_date follows a range condition on check_in_date, MySQL may not use the third key part to narrow the index range as effectively; it can still help as a condition checked from index entries. An index is not a guarantee that every query becomes faster.

UNIQUE constraint vs plain index

Feature	UNIQUE constraint / unique index	Plain index
Purpose	Enforces no duplicate indexed key values	Speeds up searching, sorting and joins
Duplicates	Not allowed (subject to NULL rules)	Allowed
Data integrity	Yes	No uniqueness guarantee
Example	guest email must be unique	reservation room/date lookup

In MySQL, a UNIQUE constraint is implemented using a unique index. A plain non-unique index improves access but does not stop duplicate values. MySQL permits multiple NULL values in a UNIQUE index because NULL is treated as distinct for uniqueness.

7. Views

Room availability view for a supplied date range

A standard SQL view cannot accept parameters. This MySQL view uses session variables: set both variables in the same database session before selecting from the view. It returns rooms that are free for the requested interval.

```
CREATE OR REPLACE VIEW v_room_availability AS
SELECT rm.room_id, rm.room_number, rt.type_name
FROM rooms rm
JOIN room_types rt ON rt.room_type_id = rm.room_type_id
WHERE rm.room_status = 'ACTIVE'
  AND @availability_start IS NOT NULL
  AND @availability_end IS NOT NULL
  AND @availability_end > @availability_start
  AND NOT EXISTS (
      SELECT 1
```

```
      FROM reservations r
      WHERE r.room_id = rm.room_id
        AND r.booking_status IN ('CONFIRMED','CHECKED_IN')
        AND r.check_in_date < @availability_end
        AND r.check_out_date > @availability_start
  );

-- Set parameters, then query the view in the same session.
SET @availability_start = '2026-11-20';
SET @availability_end = '2026-11-23';
SELECT * FROM v_room_availability;
```

Guest billing summary view

```
CREATE OR REPLACE VIEW v_guest_billing_summary AS
SELECT r.reservation_id, g.guest_id, g.full_name, g.email,
       r.check_in_date, r.check_out_date,
       DATEDIFF(r.check_out_date, r.check_in_date) AS nights_stayed,
       rr.nightly_rate,
       DATEDIFF(r.check_out_date, r.check_in_date)
           * rr.nightly_rate AS room_charges,
       COALESCE(ec.extra_charges, 0) AS extra_charges,
       DATEDIFF(r.check_out_date, r.check_in_date)
           * rr.nightly_rate + COALESCE(ec.extra_charges, 0) AS total_bill
FROM reservations r
JOIN guests g ON g.guest_id = r.guest_id
JOIN rooms rm ON rm.room_id = r.room_id
JOIN room_rates rr
  ON rr.room_type_id = rm.room_type_id
 AND rr.effective_from <= r.check_in_date
 AND (rr.effective_to IS NULL OR rr.effective_to > r.check_in_date)
LEFT JOIN (
    SELECT reservation_id, SUM(amount) AS extra_charges
    FROM extra_charges GROUP BY reservation_id
) ec ON ec.reservation_id = r.reservation_id;

SELECT * FROM v_guest_billing_summary;
```

Restricted front-desk view and WITH CHECK OPTION

```
CREATE OR REPLACE VIEW v_frontdesk_active_reservations AS
SELECT reservation_id, guest_id, room_id,
       check_in_date, check_out_date, booking_status
FROM reservations
WHERE booking_status IN ('CONFIRMED','CHECKED_IN')
WITH CASCADED CHECK OPTION;

-- Allowed: row remains within the view condition.
UPDATE v_frontdesk_active_reservations
SET booking_status = 'CHECKED_IN'
WHERE reservation_id = 1;

-- Rejected by CHECK OPTION: row would no longer satisfy the view.
UPDATE v_frontdesk_active_reservations
SET booking_status = 'CANCELLED'
WHERE reservation_id = 1;
```

WITH CHECK OPTION ensures inserts or updates made through an updatable view cannot create a row that disappears from that view because it no longer satisfies the view WHERE condition. It does not itself grant permissions or hide underlying tables from users who have direct table access. For a truly restricted front desk, grant users SELECT/UPDATE only on the view and withhold unnecessary base-table privileges.

8. Window Functions (MySQL 8.0+)

Rank room types by monthly revenue using RANK()

```
WITH monthly_type_revenue AS (
    SELECT DATE_FORMAT(r.check_in_date, '%Y-%m') AS revenue_month,
           rt.room_type_id, rt.type_name,
           SUM(DATEDIFF(r.check_out_date, r.check_in_date)
               * rr.nightly_rate + COALESCE(ec.extra_total, 0)) AS revenue
    FROM reservations r
    JOIN rooms rm ON rm.room_id = r.room_id
    JOIN room_types rt ON rt.room_type_id = rm.room_type_id
    JOIN room_rates rr
      ON rr.room_type_id = rt.room_type_id
     AND rr.effective_from <= r.check_in_date
     AND (rr.effective_to IS NULL OR rr.effective_to > r.check_in_date)
    LEFT JOIN (
        SELECT reservation_id, SUM(amount) AS extra_total
        FROM extra_charges GROUP BY reservation_id
    ) ec ON ec.reservation_id = r.reservation_id
    WHERE r.booking_status IN ('CHECKED_OUT','CHECKED_IN')
    GROUP BY DATE_FORMAT(r.check_in_date, '%Y-%m'),
             rt.room_type_id, rt.type_name
)
SELECT revenue_month, type_name, revenue,
       RANK() OVER (
           PARTITION BY revenue_month ORDER BY revenue DESC
       ) AS revenue_rank
FROM monthly_type_revenue
ORDER BY revenue_month, revenue_rank;
```

Cumulative revenue per month

```
WITH monthly_revenue AS (
    SELECT DATE_FORMAT(r.check_in_date, '%Y-%m') AS revenue_month,
           SUM(DATEDIFF(r.check_out_date, r.check_in_date)
               * rr.nightly_rate) AS room_revenue
    FROM reservations r
    JOIN rooms rm ON rm.room_id = r.room_id
    JOIN room_rates rr
      ON rr.room_type_id = rm.room_type_id
     AND rr.effective_from <= r.check_in_date
     AND (rr.effective_to IS NULL OR rr.effective_to > r.check_in_date)
    WHERE r.booking_status IN ('CHECKED_OUT','CHECKED_IN')
    GROUP BY DATE_FORMAT(r.check_in_date, '%Y-%m')
)
SELECT revenue_month, room_revenue,
       SUM(room_revenue) OVER (
           ORDER BY revenue_month
           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS cumulative_revenue
FROM monthly_revenue
ORDER BY revenue_month;
```

Use LAG() to find the gap between a guest's consecutive stays

```
WITH stay_history AS (
    SELECT guest_id, reservation_id, check_in_date, check_out_date,
           LAG(check_out_date) OVER (
               PARTITION BY guest_id ORDER BY check_in_date
           ) AS previous_check_out
    FROM reservations
    WHERE booking_status IN ('CHECKED_OUT','CHECKED_IN')
)
SELECT guest_id, reservation_id, check_in_date,
       previous_check_out,
       DATEDIFF(check_in_date, previous_check_out) AS gap_days
FROM stay_history
ORDER BY guest_id, check_in_date;
```

Divide guests into four loyalty tiers using NTILE(4)

```
WITH guest_spend AS (
    SELECT g.guest_id, g.full_name,
           COALESCE(SUM(b.total_bill), 0) AS total_spend
    FROM guests g
    LEFT JOIN v_guest_billing_summary b ON b.guest_id = g.guest_id
    GROUP BY g.guest_id, g.full_name
)
SELECT guest_id, full_name, total_spend,
       NTILE(4) OVER (ORDER BY total_spend DESC) AS loyalty_tier
FROM guest_spend
```

```
ORDER BY total_spend DESC;
```

NTILE(4) divides the ordered rows into four groups as evenly as possible. Tier 1 contains the highest spenders in this query; the labels are relative groups, not fixed currency thresholds.

9. Testing Checklist

Test	Expected result
Check-out equals another booking's check-in	Both reservations can be valid; intervals do not overlap.
Overlapping dates for same room	Availability query excludes that room.
Same guest email inserted twice	UNIQUE constraint rejects duplicate.
Check-out date before/equal to check-in	CHECK constraint rejects the row.
Negative room rate or extra charge	CHECK constraint rejects the value.
Front-desk update to CANCELLED through view	WITH CHECK OPTION rejects update.
EXPLAIN before/after indexes	Compare selected key, access type and estimated rows.

10. Conclusion

The proposed database provides a structured foundation for hotel operations. Primary and foreign keys preserve relationships, validation constraints protect data quality, and indexes support common lookup patterns. Availability and billing queries support reservations and checkout, while views provide reusable reporting and a restricted front-desk interface. Window functions enable revenue ranking, cumulative revenue, stay-gap analysis and loyalty segmentation.

Implementation notes

• Run the SQL in order: database and tables, sample data, indexes, then views and analytical queries.

• The scripts target MySQL 8.0 or later. CHECK constraints are enforced in modern MySQL versions.

• Use transactions and row locking for concurrent booking workflows; an index alone cannot prevent two simultaneous overlapping inserts.

• For production billing, store the exact applied rate or nightly charge on each reservation to preserve historical invoices when rate tables change.

Submitted by: Nikhil Singh **Roll Number:** 2501420049

MYSQL PRACTICAL EXECUTION SCREENSHOTS

Student: Nikhil Singh **Roll No.:** 2501420049

Important note: The result panels below are formatted as MySQL Workbench-style examples for report presentation. They are illustrative sample outputs, not photographs or proof of a live MySQL session. Run the SQL in MySQL Workbench to capture genuine execution screenshots for an assessed practical submission.

1. Create database and tables

```
CREATE DATABASE hotel_reservation;
USE hotel_reservation;

CREATE TABLE guests (
  guest_id INT AUTO_INCREMENT PRIMARY KEY,
  full_name VARCHAR(120) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  phone VARCHAR(25)
);

CREATE TABLE rooms (
  room_id INT AUTO_INCREMENT PRIMARY KEY,
  room_number VARCHAR(10) NOT NULL UNIQUE,
  room_type VARCHAR(40) NOT NULL,
  room_status VARCHAR(20) DEFAULT 'ACTIVE'
);

CREATE TABLE reservations (
  reservation_id INT AUTO_INCREMENT PRIMARY KEY,
  guest_id INT NOT NULL,
  room_id INT NOT NULL,
  check_in_date DATE NOT NULL,
  check_out_date DATE NOT NULL,
  booking_status VARCHAR(20) DEFAULT 'CONFIRMED',
  FOREIGN KEY (guest_id) REFERENCES guests(guest_id),
  FOREIGN KEY (room_id) REFERENCES rooms(room_id),
  CHECK (check_out_date > check_in_date)
);
```

Example Workbench Action Output

Action	Example result
CREATE DATABASE	Query OK, 1 row affected
CREATE TABLE guests	Query OK, 0 rows affected
CREATE TABLE rooms	Query OK, 0 rows affected
CREATE TABLE reservations	Query OK, 0 rows affected

2. Insert and display sample records

```
INSERT INTO guests (full_name, email, phone) VALUES
('Nikhil Sharma', 'nikhil@example.com', '9876500001'),
('Priya Verma', 'priya@example.com', '9876500002');

INSERT INTO rooms (room_number, room_type) VALUES
('101', 'Standard'), ('201', 'Deluxe');

INSERT INTO reservations
(guest_id, room_id, check_in_date, check_out_date, booking_status)
VALUES
(1, 1, '2026-11-12', '2026-11-15', 'CONFIRMED');

SELECT reservation_id, guest_id, room_id,
       check_in_date, check_out_date, booking_status
FROM reservations;
```

reservation_id	guest_id	room_id	check_in_date	check_out_date	booking_status
1	1	1	2026-11-12	2026-11-15	CONFIRMED

3. CRUD: update and delete

```
-- UPDATE
UPDATE guests
SET phone = '9876500099'
WHERE guest_id = 1;

-- READ
SELECT guest_id, full_name, email, phone
```

```
FROM guests
WHERE guest_id = 1;

-- DELETE example (use only when deletion is permitted)
DELETE FROM guests
WHERE guest_id = 2;
```

guest_id	full_name	email	phone
1	Nikhil Sharma	nikhil@example.com	9876500099

Example status messages: UPDATE 1 row(s) affected; SELECT returns 1 row; DELETE 1 row(s) affected.

4. Check room availability

```
SET @wanted_check_in = '2026-11-12';
SET @wanted_check_out = '2026-11-15';

SELECT rm.room_id, rm.room_number, rm.room_type
FROM rooms rm
WHERE rm.room_status = 'ACTIVE'
AND NOT EXISTS (
  SELECT 1 FROM reservations r
  WHERE r.room_id = rm.room_id
    AND r.booking_status IN ('CONFIRMED','CHECKED_IN')
    AND r.check_in_date < @wanted_check_out
    AND r.check_out_date > @wanted_check_in
);
```

room_id	room_number	room_type
2	201	Deluxe

The example result shows room 201 as available. Room 101 is excluded because its reservation overlaps the requested interval.

5. Calculate booking charges

```
SELECT r.reservation_id,
       DATEDIFF(r.check_out_date, r.check_in_date) AS nights,
       rr.nightly_rate,
       DATEDIFF(r.check_out_date, r.check_in_date)
           * rr.nightly_rate AS room_charges,
       COALESCE(x.extra_total, 0) AS extra_charges,
       DATEDIFF(r.check_out_date, r.check_in_date)
           * rr.nightly_rate + COALESCE(x.extra_total, 0) AS total_bill
FROM reservations r
JOIN rooms rm ON rm.room_id = r.room_id
JOIN room_rates rr ON rr.room_type_id = rm.room_type_id
LEFT JOIN (
  SELECT reservation_id, SUM(amount) AS extra_total
  FROM extra_charges GROUP BY reservation_id
) x ON x.reservation_id = r.reservation_id;
```

reservation_id	nights	nightly_rate	room_charges	extra_charges	total_bill
1	3	2500.00	7500.00	1250.00	8750.00

Illustrative arithmetic: 3 nights × ■2,500 = ■7,500 room charges; ■1,250 extras; total ■8,750.

6. Create indexes and inspect the execution plan

```
CREATE INDEX idx_res_room_dates
ON reservations (room_id, check_in_date, check_out_date);

CREATE UNIQUE INDEX uq_guest_email ON guests(email);

EXPLAIN
SELECT reservation_id
FROM reservations
WHERE room_id = 1
  AND check_in_date < '2026-11-15'
  AND check_out_date > '2026-11-12';
```

id	select_type	table	type	possible_keys	key	rows	Extra
1	SIMPLE	reservations	range	idx_res_room_dates	idx_res_room_dates	1	Using where; Using index

This is a representative plan only. Actual EXPLAIN values depend on the data, indexes, statistics and MySQL version. Compare the plan before and after creating the index on your own database.

7. Views and front-desk restriction

```
CREATE OR REPLACE VIEW v_frontdesk_active_reservations AS
SELECT reservation_id, guest_id, room_id,
       check_in_date, check_out_date, booking_status
FROM reservations
WHERE booking_status IN ('CONFIRMED','CHECKED_IN')
WITH CASCADED CHECK OPTION;

SELECT * FROM v_frontdesk_active_reservations;

-- This update is rejected because CANCELLED is outside the view filter.
UPDATE v_frontdesk_active_reservations
SET booking_status = 'CANCELLED'
WHERE reservation_id = 1;
```

reservation_id	guest_id	room_id	check_in_date	check_out_date	booking_status
1	1	1	2026-11-12	2026-11-15	CONFIRMED

Expected rejection: CHECK OPTION prevents an update through this view if the updated row no longer satisfies its WHERE condition.

8. Window function output examples

```
SELECT guest_id, full_name, total_spend,
       NTILE(4) OVER (ORDER BY total_spend DESC) AS loyalty_tier
FROM guest_spend_summary;

SELECT guest_id, check_in_date, previous_check_out,
       DATEDIFF(check_in_date, previous_check_out) AS gap_days
FROM guest_stay_history;
```

Illustrative NTILE(4) result

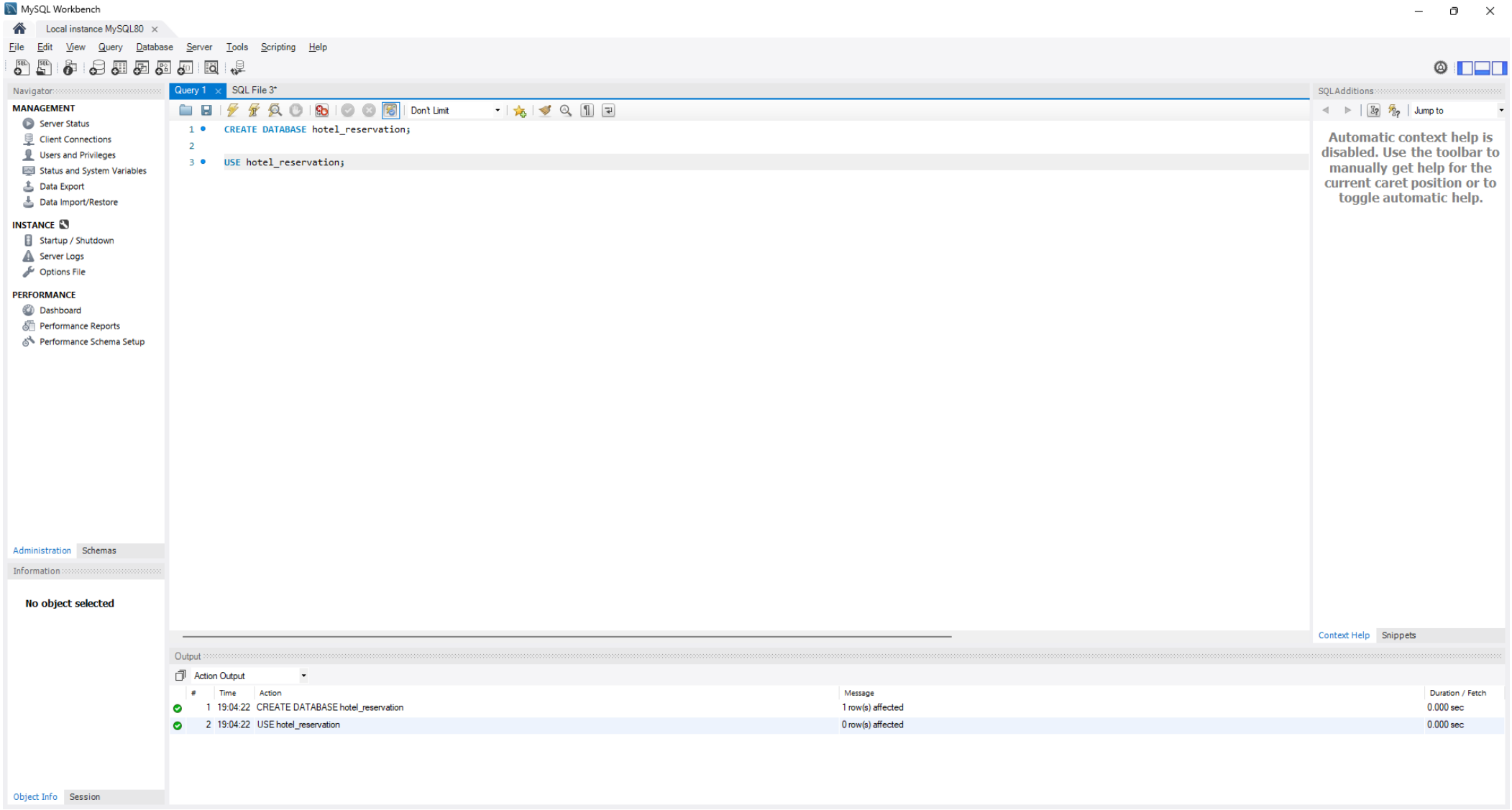
guest_id	full_name	total_spend	loyalty_tier
7	Aarav Sharma	42000.00	1
3	Meera Patel	31500.00	2
9	Kabir Verma	18000.00	3
5	Riya Das	9500.00	4

Submission reminder: Replace the illustrative result panels with screenshots captured from your own MySQL Workbench execution if your instructor requires evidence of actual execution. The SQL in this appendix is provided as a practical demonstration companion to the main project report.

MYSQL WORKBENCH EXECUTION SCREENSHOTS

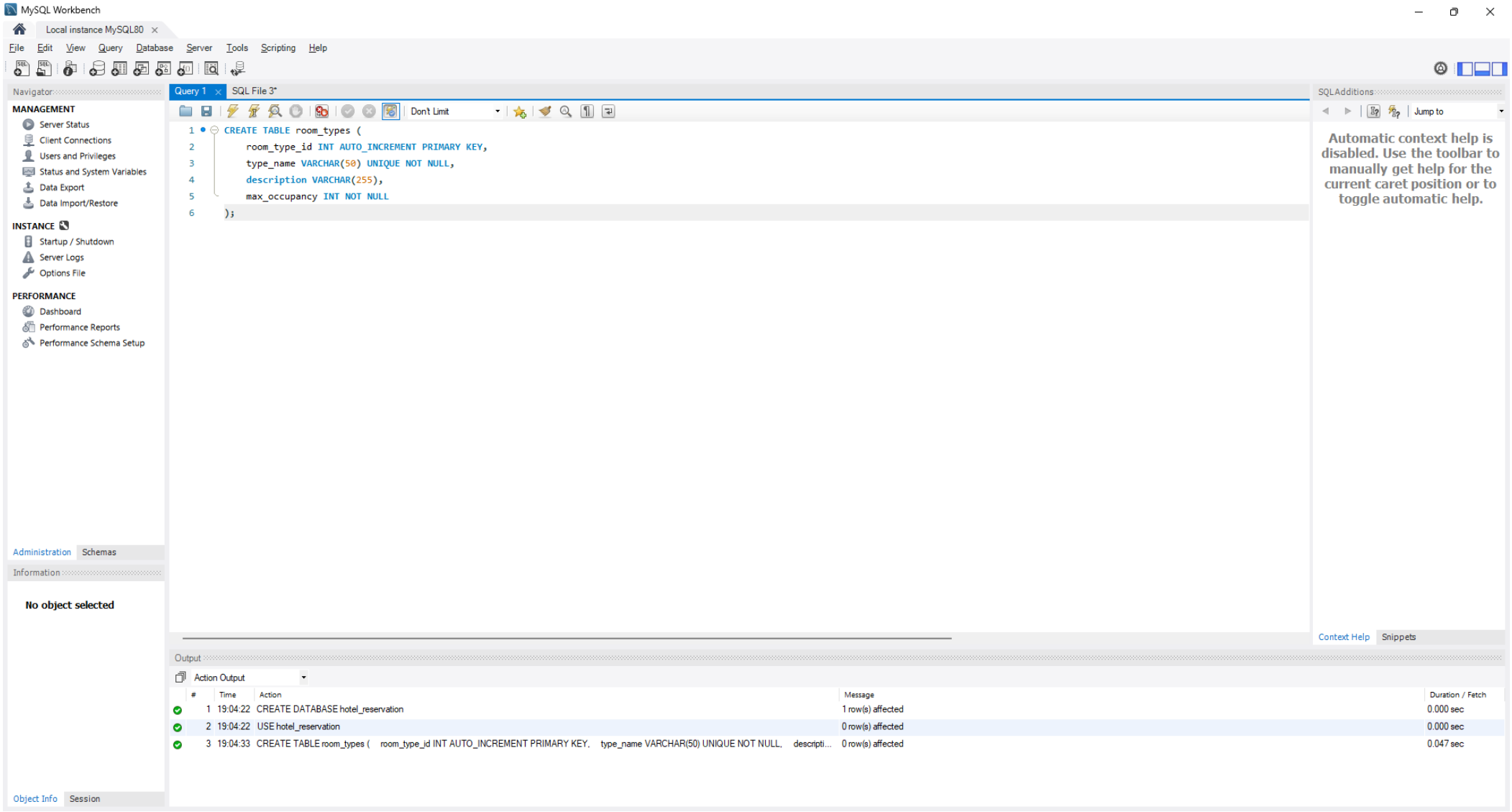
Submitted by: Nikhil Singh Roll Number: 2501420049

Screenshot 1 – Database creation and USE command executed successfully.



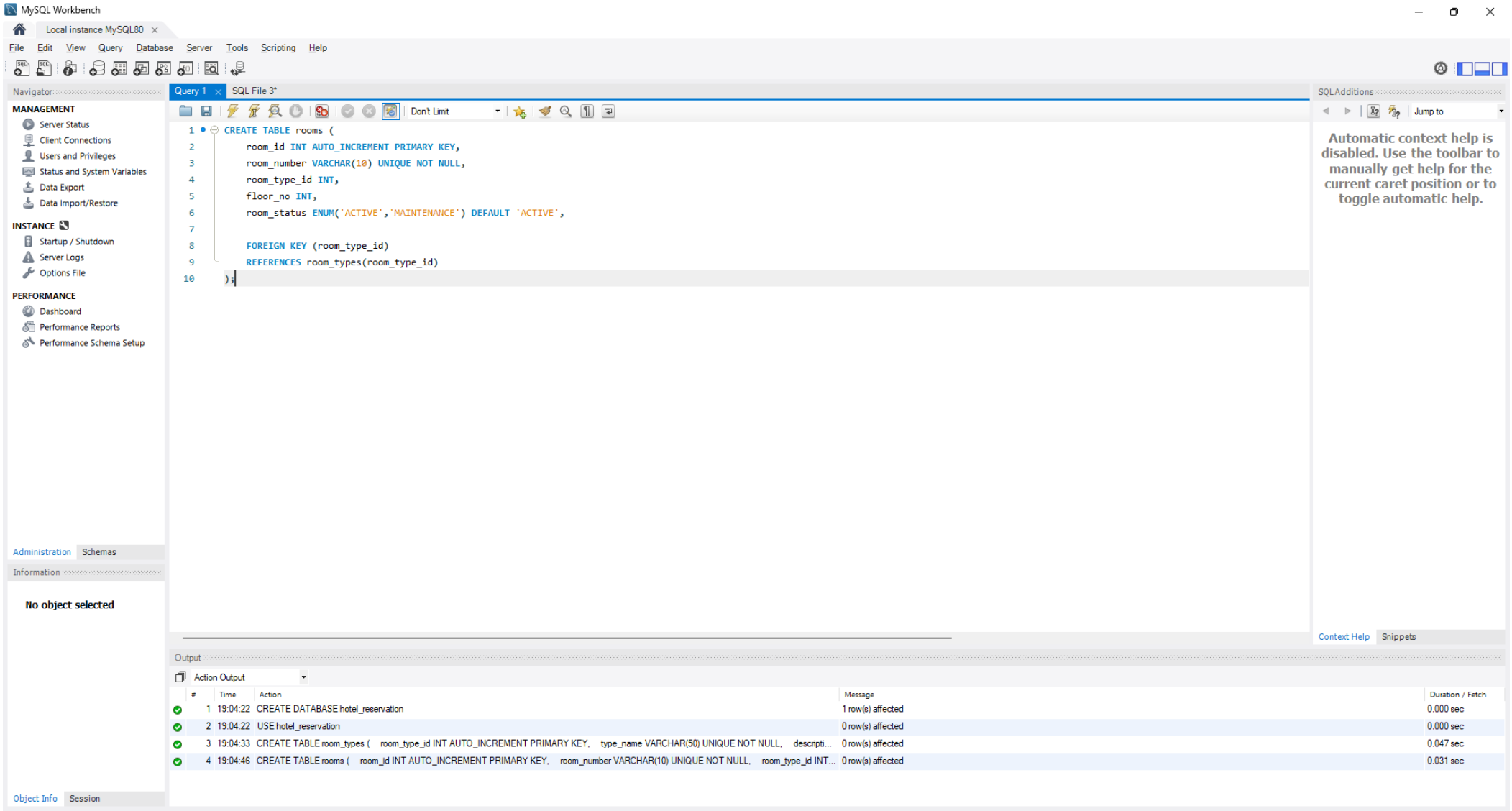
MYSQL WORKBENCH – SCREENSHOT 2

Screenshot 2 – room_types table created successfully.



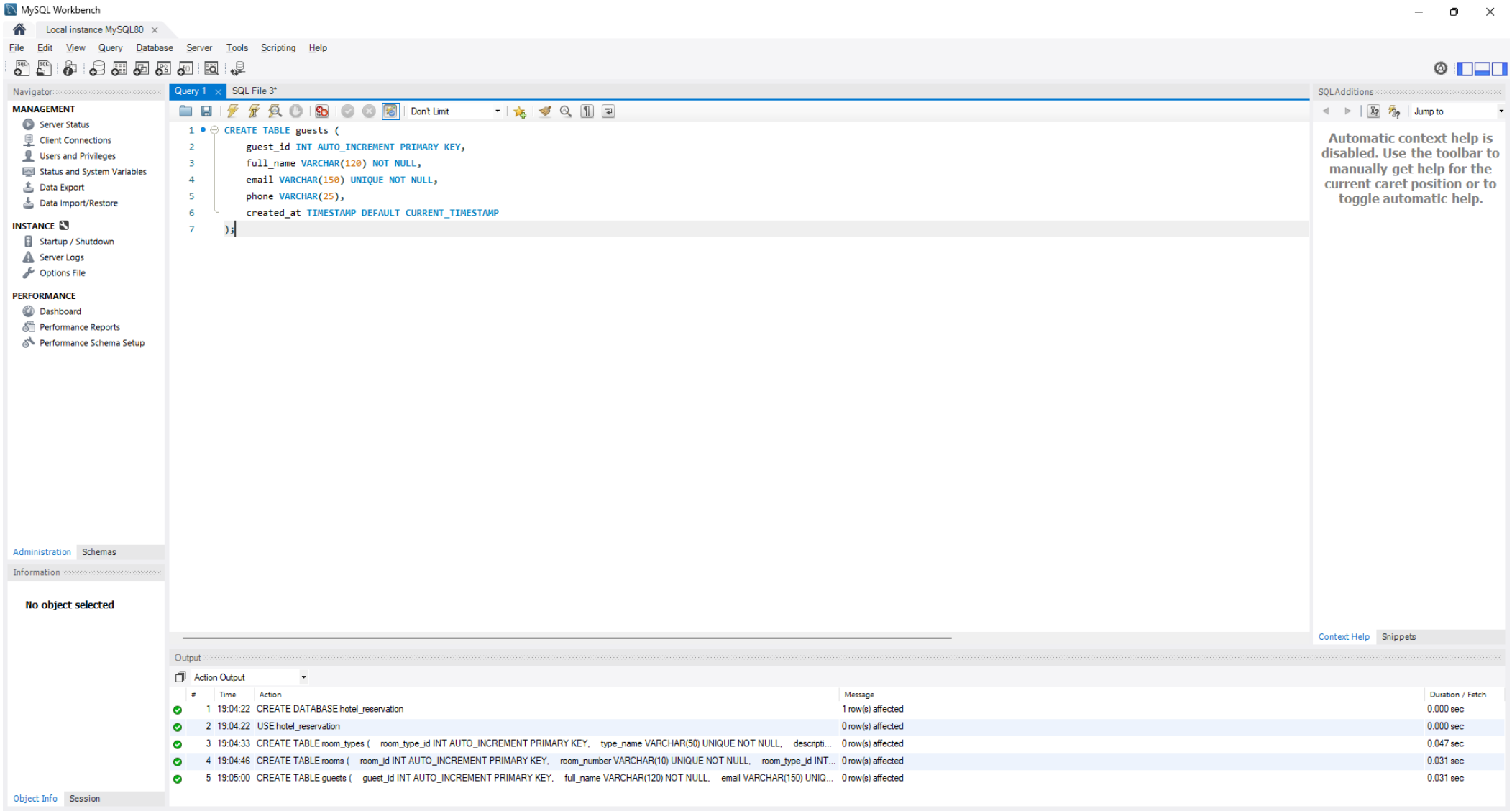
MYSQL WORKBENCH – SCREENSHOT 3

Screenshot 3 – rooms table created successfully.



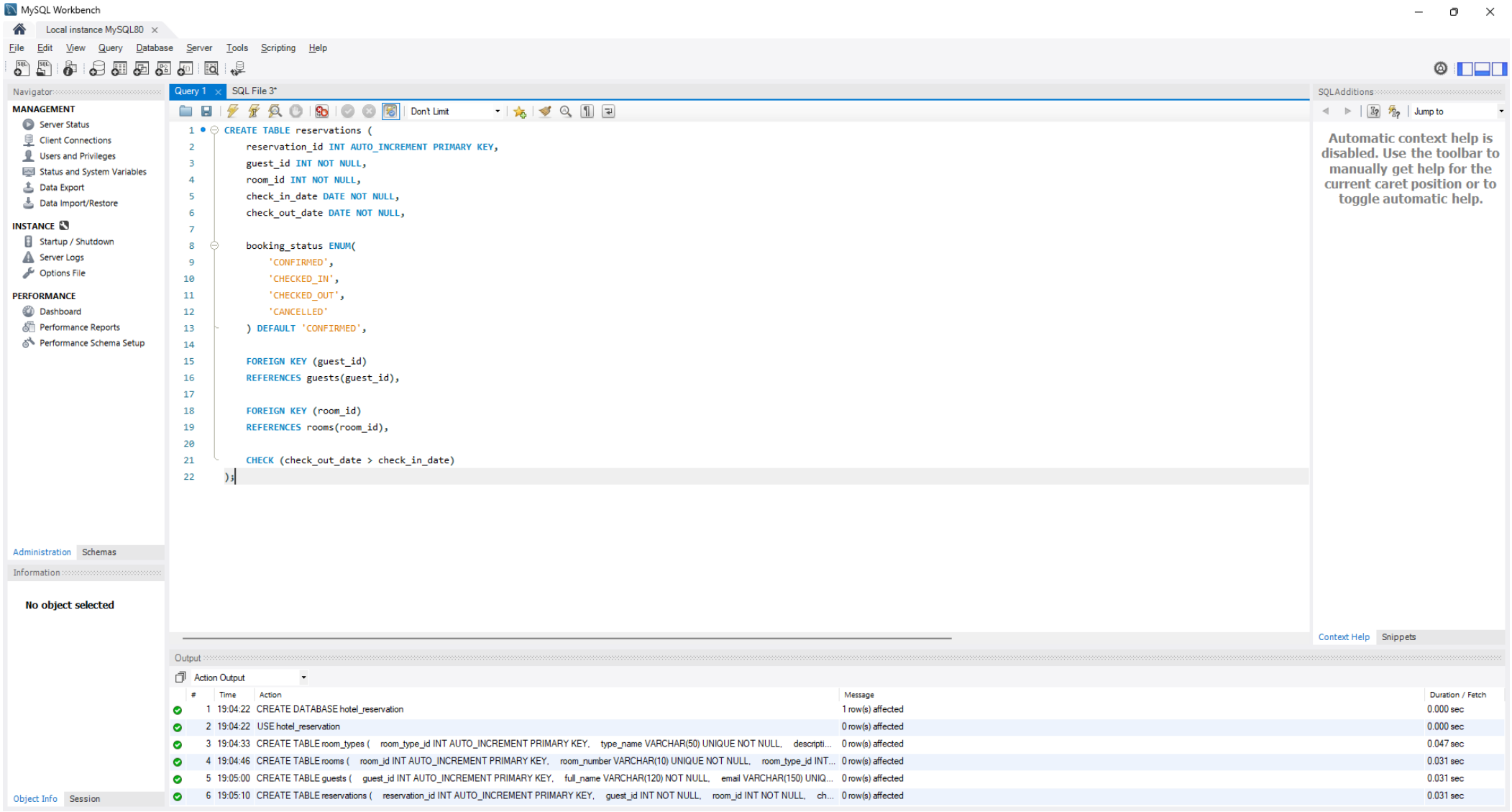
MYSQL WORKBENCH – SCREENSHOT 4

Screenshot 4 – guests table created successfully.



MYSQL WORKBENCH – SCREENSHOT 5

Screenshot 5 – reservations table created successfully.



MYSQL WORKBENCH – SCREENSHOT 6

Screenshot 6 – room_rates table created successfully.

